

Contents

1. Introduction	.2
2. Literature Review	.3
3. Objectives.....	.4
4. Methodology	.5
4.1 Architectural Diagram	.5
4.2 Hardware and Software Requirements	.6
4.3 Working Principle.....	.6
4.4 Software Implementation.....	.8
5. Results and Discussion	.24
6. Mapping POs, PSOs and SDGs	.29
6.1 Program Outcomes (POs) Mapping.....	.29
6.2 Program Specific Outcomes(PSOs) Mapping.....	.29
6.3 Sustainable Development Goals (SDGs) Mapping	.30
7. Conclusion	.31
Bibliography	.32

Abstract

The College Course Registration System is a desktop application developed in Python that automates the process of enrolling students in the courses offered by a college. The system is designed using the principles of object-oriented programming and is organised around three core classes, **Student**, **Course** and **RegistrationSystem**, together with a custom exception class, **RegistrationError**, that reports every rule violation in a uniform way. A graphical user interface built with Tkinter allows users to add students and courses, register or drop a course, and generate a registration report for any student without typing commands in a terminal.

The application enforces three academic rules automatically: a student cannot register for more than 24 credits, a course cannot enrol more students than its seat capacity, and a student cannot register for the same course twice. All data is stored in a JSON file using a safe save strategy, so registrations survive between sessions, and the loader skips duplicate or invalid records so that a damaged file does not crash the program. Keeping the logic classes separate from the GUI makes the code readable, testable and reusable. The project demonstrates how classes, encapsulation, exception handling, file handling and event-driven programming come together in a practical Python application.

Keywords: Python, Object-Oriented Programming, Tkinter, GUI, Course Registration, Encapsulation, Exception Handling, JSON, File Handling.

1. Introduction

Course registration is one of the most important administrative activities in any college. At the start of every semester, students must choose the subjects they want to study, and the institution must make sure that each choice is valid: the course must still have seats available, the student must not exceed the permitted workload, and the same course must not be taken twice. When this work is done on paper or in spreadsheets, it is slow and error-prone. Over-booked classrooms, duplicate entries and students carrying an unmanageable number of credits are common results of manual processing.

A software-based registration system removes most of these problems by checking every rule at the moment a registration is attempted. The user gets an immediate answer, and the data is always kept in a consistent state. Such systems are the backbone of student information systems used by colleges and universities, and understanding how they work is a useful exercise for any computer science student.

This project implements a simplified but complete course registration system in Python. Python was chosen because of its clear syntax, strong support for object-oriented programming, and a rich standard library that includes Tkinter for building graphical interfaces and the json module for storing data. The design uses three classes that mirror real-world entities, Student, Course and RegistrationSystem, so that the structure of the program closely follows the structure of the problem. A fourth class, RegistrationApp, provides the graphical interface and simply calls the methods of the registration system.

The purpose of the project is to apply the concepts learned in the Python course, namely classes and objects, encapsulation using properties, custom exceptions, collections, file handling and GUI programming, to build a useful application. The remainder of this report reviews related work, states the objectives, explains the architecture and working of the system, presents the source code and the results of testing, maps the project to the Program Outcomes and Sustainable Development Goals, and concludes with possible future enhancements.

2. Literature Review

Student information systems and enterprise resource planning (ERP) platforms are widely used by educational institutions to manage admissions, attendance, examinations and course enrolment. Commercial and open-source learning management systems also provide enrolment features, usually through web portals backed by relational databases. Database textbooks such as Elmasri and Navathe (2016) use university registration as a standard example because it brings out the central issues of data management: entities with attributes, relationships between them, and integrity constraints that must always hold.

A recurring theme in this domain is the enforcement of constraints. Seat limits prevent over-booking of classrooms and laboratories, credit limits protect students from excessive workload, and uniqueness rules prevent duplicate enrolment. In large systems these constraints are implemented through a combination of database rules and application logic. Software engineering literature (Sommerville, 2016) recommends placing such business rules in a dedicated layer, separate from the user interface, so that they can be tested and changed independently. Object-oriented analysis and design (Booch et al., 2007) supports this approach by modelling real-world entities as classes that hide their internal data and expose a small, well-defined set of operations.

For learners, many course registration examples are written as terminal programs with text menus or as spreadsheets with manual checks. These are easy to write but are less intuitive to use and give little insight into how a graphical, event-driven application is structured. On the other hand, full web-based systems require knowledge of databases, servers and front-end frameworks, which is beyond the scope of an introductory Python course.

This project addresses that gap. It provides a graphical desktop application that needs only the Python standard library, applies a clean object-oriented design with validation and custom exceptions, and stores its data in a human-readable JSON file instead of a database server. It therefore serves as a practical bridge between classroom concepts and real application development, and it can later be extended with a database or a web interface without changing the core classes.

3. Objectives

To design and develop a Python-based course registration system using object-oriented programming.

The first objective is to build a working course registration system whose structure follows the real-world problem. Students, courses and the registration process are modelled by the classes Student, Course and RegistrationSystem. Each class keeps its data private and exposes it through properties and methods, demonstrating encapsulation, while a custom RegistrationError exception communicates every invalid operation in a consistent way.

To implement a Tkinter-based GUI for managing students, courses, registrations and reports.

A graphical interface makes the system usable by people who are not programmers. This objective covers a tabbed window with separate tabs for adding students, adding courses, registering or dropping a course, and generating a registration report. The interface uses forms, drop-down lists, tables and message boxes so that users always see the current state of the data and receive a clear success or error message after each action.

To enforce registration rules and provide persistent storage of data.

The system must automatically enforce the maximum credit limit of 24 per student, the seat capacity of every course, and the rule against duplicate registration. In addition, all students, courses and registrations should be saved to a JSON file after every change and restored when the program starts again, with safeguards against missing or damaged files. Together, these features show how file handling and exception handling help build a reliable application.

4. Methodology

4.1 Architectural Diagram

The system is organised into three layers, as shown in Figure 1. The presentation layer is the Tkinter GUI, the business logic layer contains the classes that represent the problem and its rules, and the persistence layer is a JSON file. Dividing the program in this way means that the GUI never changes data directly; it always goes through the methods of RegistrationSystem.

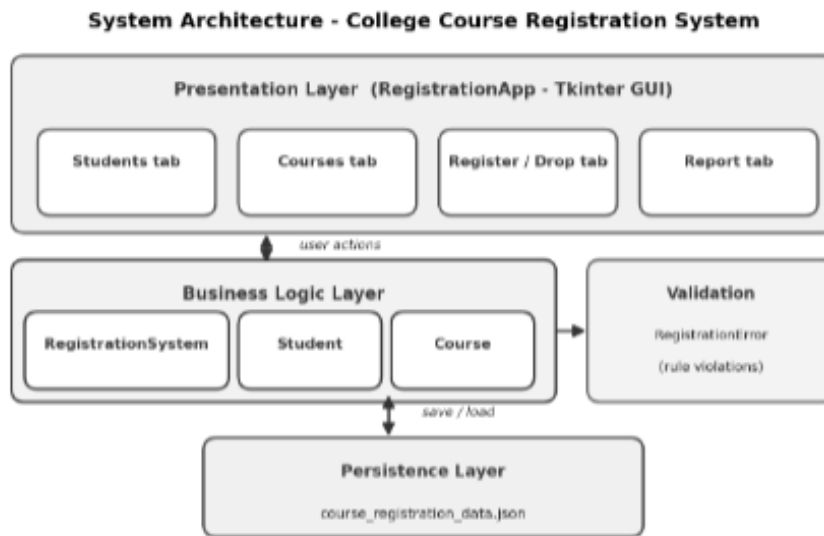


Figure 1: Architectural Diagram

The **Presentation Layer** is the RegistrationApp class, which extends tk.Tk. It creates a notebook with four tabs (Students, Courses, Register / Drop and Report), reads the values typed or selected by the user, calls the registration system, and displays the outcome using tables and message boxes.

The **Business Logic Layer** consists of RegistrationSystem, Student and Course. RegistrationSystem holds a dictionary of students and a dictionary of courses and carries out all operations (add, register, drop, save and load). Student and Course store their own data and check the rules that concern them, for example Student.can_take() checks the credit limit and Course.has_seat() checks the capacity.

The **Validation** component is the RegistrationError exception. Every class raises it when an input or a rule is violated, and the GUI catches it in one place and shows the message to the user.

The **Persistence Layer** is the file course_registration_data.json, which stores the lists of students, courses and registrations. The file is written after every successful change and read once at start-up.

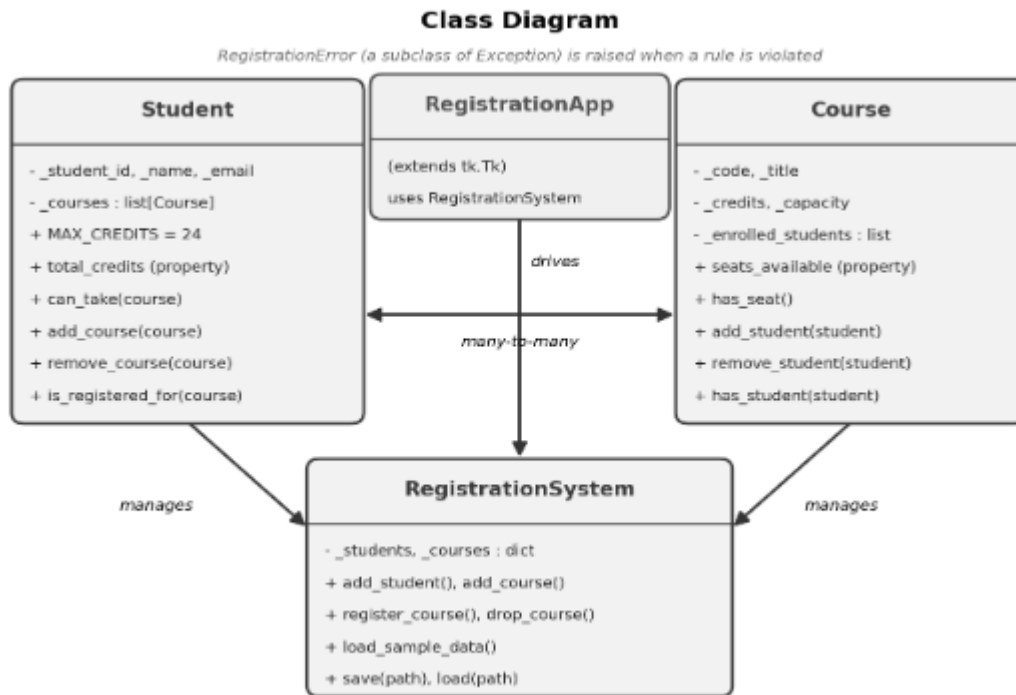


Figure 2: Class Diagram

Figure 2 shows the relationships between the classes. A student can be registered for many courses and a course can have many students, so the relationship is many-to-many. Each side keeps a list of the other, and RegistrationSystem is responsible for updating both lists together so that they never get out of step.

4.2 Hardware and Software Requirements

Hardware:

- Intel/AMD processor
- Minimum 4 GB RAM
- Standard display, keyboard and mouse

Software:

- Python 3.8 or later (Tkinter is included with the standard Python installer; on some Linux systems it must be installed separately as python3-tk)
- Python standard library modules only: tkinter, json and pathlib (no third-party packages required)
- Any code editor or IDE (IDLE, VS Code or PyCharm)
- Operating system: Windows, Linux or macOS

4.3 Working Principle

The system works by modelling the registration process exactly as it happens in a college. Every student and every course is an object. When a student registers for a course, the system checks a fixed

sequence of conditions and, only if all of them are satisfied, links the two objects together. The working of the application can be summarised in the following steps.

1. **Start-up.** RegistrationApp looks for the file `course_registration_data.json` next to the program. If it exists, `RegistrationSystem.load()` rebuilds all students, courses and registrations from it. If it does not exist, or cannot be read, a new system is created, filled with sample data and saved.
2. **Adding students and courses.** The user fills in a form and presses the button. The Student or Course constructor validates the input (non-empty ID and name, valid e-mail, positive credits and capacity) and the system rejects duplicate IDs or course codes. On success the data is saved and the tables and drop-down lists are refreshed.
3. **Registering for a course.** The user selects a student and a course from the drop-down lists, which show entries in the form “ID - Name”. The ID is extracted and `register_course()` checks, in order: the student exists, the course exists, the student is not already registered, a seat is available, and the credit limit would not be exceeded. If all checks pass, both the course and the student are updated.
4. **Dropping a course.** `drop_course()` verifies that the student is registered for the course and then removes the link from both objects.
5. **Saving.** After each successful change the whole state is written to the JSON file. The data is first written to a temporary file which then replaces the real file, so the saved file is never left half-written.
6. **Reporting.** For a selected student the report tab prints a formatted statement listing the registered courses, the total credits, the credit limit and the remaining credits.

Table 1: Business rules enforced by the system

Rule	Enforced by	Message shown to the user (example)
Maximum 24 credits per student	<code>Student.can_take()</code>	Credit limit exceeded. Current: 24, Course: 3, Max allowed: 24.
Seat capacity of a course	<code>Course.has_seat()</code>	CS101 is full. No seats available.
No duplicate registration	<code>Student.is_registered_for()</code>	Aarav Sharma is already registered for CS101.
Unique student ID / course code	<code>RegistrationSystem.add_student()</code> , <code>add_course()</code>	Student ID 'S101' already exists.
Valid input data	Student and Course constructors	Please enter a valid email address.

Example problem. The sample data loaded by the application contains three students and 38 courses. Two courses are deliberately small so that the capacity rule can be seen quickly: CS101 (4 credits, 3 seats) and CS201 (4 credits, 2 seats). The initial registrations are shown in Table 2.

Table 2: Initial registrations in the sample data

Student ID	Name	Registered courses	Credits used
S101	Aarav Sharma	CS101, CS201	8 / 24
S102	Diya Reddy	CS101	4 / 24
S103	Rohan Mehta	None	0 / 24

Now suppose S101 also registers for CS401 (6 credits), CS402 (6 credits) and CS505 (4 credits). The total grows to 14, then 20, then 24 credits, and every registration is accepted because the limit is not exceeded. If S101 then tries to register for CS504 (3 credits), the system calculates $24 + 3 = 27$, which is greater than 24, and rejects the request with a credit limit message. Similarly, once S103 takes the last seat in CS101 (capacity 3), any further student who tries to register for CS101 is told that the course is full. The results of such tests are listed in Chapter 5.

4.4 Software Implementation

The complete program is written in a single Python file and uses only the standard library. The responsibilities of the classes are summarised in Table 3, and the full source code follows.

Table 3: Classes and their responsibilities

Class	Responsibility
RegistrationError	Custom exception raised when a rule is violated or an input is invalid.
Course	Stores code, title, credits and capacity; tracks enrolled students; checks seat availability.
Student	Stores ID, name and e-mail; tracks registered courses; calculates total credits and checks the 24-credit limit.
RegistrationSystem	Manages all students and courses; adds, registers and drops; loads sample data; saves to and loads from JSON.
RegistrationApp	Tkinter window with four tabs; connects user actions to RegistrationSystem and displays results.

Source code: course_registration.py

```

"""
College Course Registration System - GUI Version
=====
Same OOP design as the terminal version (Student, Course, RegistrationSystem),
with a Tkinter graphical interface layered on top instead of a text menu.

Classes
-----
Student      -> manages student data and the student's registered courses
Course       -> manages course data and the course's enrolled students
RegistrationSystem -> manages all operations (add, register, drop, report)

```

RegistrationApp -> the Tkinter GUI that drives RegistrationSystem

Rules enforced

- * A student cannot exceed 24 credits.
- * A course cannot exceed its seat capacity.
- * A student cannot register for the same course twice.

"""

```
import json
import tkinter as tk
from tkinter import ttk, messagebox
from pathlib import Path
```

```
class RegistrationError(Exception):
    """Raised when a registration rule is violated or input is invalid."""
```

```
# -----
```

```
# Course
```

```
# -----
```

```
class Course:
    """Stores course data and tracks which students are enrolled."""
```

```
    def __init__(self, code, title, credits, capacity):
        if credits <= 0:
            raise RegistrationError("Credits must be a positive number.")
        if capacity <= 0:
            raise RegistrationError("Capacity must be a positive number.")
        self._code = code.strip().upper()
        self._title = title.strip()
        self._credits = credits
        self._capacity = capacity
        self._enrolled_students = [] # list of Student objects
```

```
    @property
    def code(self):
        return self._code
```

```
    @property
    def title(self):
        return self._title
```

```
    @property
    def credits(self):
        return self._credits
```

```

@property
def capacity(self):
    return self._capacity

@property
def enrolled_count(self):
    return len(self._enrolled_students)

@property
def seats_available(self):
    return self._capacity - len(self._enrolled_students)

def has_seat(self):
    return self.seats_available > 0

def has_student(self, student):
    return student in self._enrolled_students

def add_student(self, student):
    if not self.has_seat():
        raise RegistrationError(f'{self._code} is full. No seats available.')
    if self.has_student(student):
        raise RegistrationError(f'Student is already enrolled in {self._code}.')
    self._enrolled_students.append(student)

def remove_student(self, student):
    if student not in self._enrolled_students:
        raise RegistrationError(f'Student is not enrolled in {self._code}.')
    self._enrolled_students.remove(student)

# -----
# Student
# -----
class Student:
    """Stores student data and the list of courses the student is registered in."""

    MAX_CREDITS = 24

    def __init__(self, student_id, name, email):
        if not student_id.strip():
            raise RegistrationError("Student ID cannot be empty.")
        if not name.strip():
            raise RegistrationError("Student name cannot be empty.")
        if "@" not in email or "." not in email:
            raise RegistrationError("Please enter a valid email address.")
        self._student_id = student_id.strip().upper()
        self._name = name.strip()

```

```

        self._email = email.strip()
        self._courses = [] # list of Course objects

    @property
    def student_id(self):
        return self._student_id

    @property
    def name(self):
        return self._name

    @property
    def email(self):
        return self._email

    @property
    def courses(self):
        return list(self._courses)

    @property
    def total_credits(self):
        return sum(course.credits for course in self._courses)

    def is_registered_for(self, course):
        return course in self._courses

    def can_take(self, course):
        if self.total_credits + course.credits > self.MAX_CREDITS:
            raise RegistrationError(
                f"Credit limit exceeded. Current: {self.total_credits}, "
                f"Course: {course.credits}, Max allowed: {self.MAX_CREDITS}."
            )

    def add_course(self, course):
        self._courses.append(course)

    def remove_course(self, course):
        self._courses.remove(course)

# -----
# RegistrationSystem
# -----
class RegistrationSystem:
    """Manages all students, courses, and registration operations."""

    def __init__(self):

```

```

self._students = {} # student_id -> Student
self._courses = {} # course_code -> Course

def _get_student(self, student_id):
    sid = student_id.strip().upper()
    if sid not in self._students:
        raise RegistrationError(f"No student found with ID '{sid}'.")
    return self._students[sid]

def _get_course(self, code):
    c = code.strip().upper()
    if c not in self._courses:
        raise RegistrationError(f"No course found with code '{c}'.")
    return self._courses[c]

@property
def students(self):
    return list(self._students.values())

@property
def courses(self):
    return list(self._courses.values())

def add_student(self, student_id, name, email):
    student = Student(student_id, name, email)
    if student.student_id in self._students:
        raise RegistrationError(f"Student ID '{student.student_id}' already exists.")
    self._students[student.student_id] = student
    return student

def add_course(self, code, title, credits, capacity):
    course = Course(code, title, credits, capacity)
    if course.code in self._courses:
        raise RegistrationError(f"Course code '{course.code}' already exists.")
    self._courses[course.code] = course
    return course

def register_course(self, student_id, course_code):
    student = self._get_student(student_id)
    course = self._get_course(course_code)

    if student.is_registered_for(course):
        raise RegistrationError(f"{student.name} is already registered for {course.code}.")
    if not course.has_seat():
        raise RegistrationError(f"{course.code} is full. No seats available.")
    student.can_take(course)

    course.add_student(student)

```

```

student.add_course(course)
return student, course

def drop_course(self, student_id, course_code):
    student = self._get_student(student_id)
    course = self._get_course(course_code)

    if not student.is_registered_for(course):
        raise RegistrationError(f"{student.name} is not registered for {course.code}.")
    course.remove_student(student)
    student.remove_course(course)
    return student, course

def load_sample_data(self):
    self.add_student("S101", "Aarav Sharma", "aarav@college.edu")
    self.add_student("S102", "Diya Reddy", "diya@college.edu")
    self.add_student("S103", "Rohan Mehta", "rohan@college.edu")

    self.add_course("CS101", "Introduction to Programming", 4, 3)
    self.add_course("CS201", "Data Structures", 4, 2)
    self.add_course("CS301", "Database Systems", 3, 30)
    self.add_course("MA101", "Calculus I", 4, 40)
    self.add_course("PH101", "Physics I", 4, 35)
    self.add_course("EN101", "Technical Communication", 2, 50)
    self.add_course("CS401", "Advanced Algorithms", 6, 25)
    self.add_course("CS402", "Machine Learning", 6, 25)
    self.add_course("CS501", "Operating Systems", 4, 35)
    self.add_course("CS502", "Computer Networks", 4, 30)
    self.add_course("CS503", "Compiler Design", 4, 25)
    self.add_course("CS504", "Cyber Security", 3, 20)
    self.add_course("CS505", "Artificial Intelligence", 4, 25)
    self.add_course("EC201", "Digital Electronics", 4, 30)
    self.add_course("EC202", "Signals and Systems", 4, 28)
    self.add_course("EC301", "Microprocessors and Microcontrollers", 4, 28)
    self.add_course("EC302", "Communication Engineering", 3, 30)
    self.add_course("EE201", "Electrical Machines", 4, 30)
    self.add_course("EE202", "Power Systems", 4, 25)
    self.add_course("ME201", "Thermodynamics", 4, 30)
    self.add_course("ME202", "Fluid Mechanics", 4, 28)
    self.add_course("CE201", "Structural Analysis", 4, 25)
    self.add_course("CE202", "Transportation Engineering", 3, 25)
    self.add_course("CS601", "Distributed Systems", 3, 22)
    self.add_course("CS602", "Software Engineering", 3, 25)
    self.add_course("CS603", "Database Security", 3, 18)
    self.add_course("CS604", "Cloud Computing", 4, 24)
    self.add_course("CS605", "Data Mining", 3, 20)
    self.add_course("EC401", "VLSI Design", 4, 24)
    self.add_course("EC402", "Embedded Systems", 4, 26)
    self.add_course("EC403", "Wireless Communication", 3, 20)

```

```

self.add_course("EC404", "Digital Signal Processing", 4, 22)
self.add_course("EE301", "Control Systems", 4, 22)
self.add_course("EE302", "Renewable Energy", 3, 18)
self.add_course("ME301", "Manufacturing Processes", 3, 20)
self.add_course("ME302", "Machine Design", 4, 22)
self.add_course("CE301", "Geotechnical Engineering", 4, 20)
self.add_course("CE302", "Hydraulics and Hydrology", 4, 20)

self.register_course("S101", "CS101")
self.register_course("S101", "CS201")
self.register_course("S102", "CS101")

def save(self, path):
    data = {
        "students": [
            {"student_id": student.student_id, "name": student.name, "email": student.email}
            for student in self.students
        ],
        "courses": [
            {
                "code": course.code,
                "title": course.title,
                "credits": course.credits,
                "capacity": course.capacity,
            }
            for course in self.courses
        ],
        "registrations": [
            {"student_id": student.student_id, "course_code": course.code}
            for student in self.students
            for course in student.courses
        ],
    }
    path = Path(path)
    temporary_path = path.with_suffix(path.suffix + ".tmp")
    temporary_path.write_text(json.dumps(data, indent=2), encoding="utf-8")
    temporary_path.replace(path)

    @classmethod
    def load(cls, path):
        path = Path(path)
        if not path.exists():
            raise FileNotFoundError(f"Data file not found: {path}")

        try:
            data = json.loads(path.read_text(encoding="utf-8"))
        except (json.JSONDecodeError, OSError) as exc:
            raise RegistrationError(f"Could not read saved data from {path}: {exc}") from exc

```

```

system = cls()
seen_courses = set()
for course in data.get("courses", []):
    code = str(course.get("code", "")).strip()
    if not code or code in seen_courses:
        continue
    seen_courses.add(code)
    try:
        system.add_course(code, course["title"], course["credits"], course["capacity"])
    except RegistrationError:
        continue

seen_students = set()
for student in data.get("students", []):
    sid = str(student.get("student_id", "")).strip()
    if not sid or sid in seen_students:
        continue
    seen_students.add(sid)
    try:
        system.add_student(sid, student["name"], student["email"])
    except RegistrationError:
        continue

for registration in data.get("registrations", []):
    sid = registration.get("student_id")
    code = registration.get("course_code")
    if sid is None or code is None:
        continue
    try:
        system.register_course(sid, code)
    except RegistrationError:
        continue
return system

```

```

# -----
# GUI
# -----
class RegistrationApp(tk.Tk):
    """Main Tkinter application window with a tab for each operation."""

    DATA_FILE = Path(__file__).with_name("course_registration_data.json")

    def __init__(self):
        super().__init__()
        try:
            if self.DATA_FILE.exists():

```

```

        self.system = RegistrationSystem.load(self.DATA_FILE)
    else:
        self.system = RegistrationSystem()
        self.system.load_sample_data()
        self.system.save(self.DATA_FILE)
except (RegistrationError, FileNotFoundError, OSError):
    self.system = RegistrationSystem()
    self.system.load_sample_data()
    self.system.save(self.DATA_FILE)

self.title("College Course Registration System")
self.geometry("880x560")
self.minsize(760, 480)

style = ttk.Style(self)
try:
    style.theme_use("clam")
except tk.TclError:
    pass

notebook = ttk.Notebook(self)
notebook.pack(fill="both", expand=True, padx=8, pady=8)

self.students_tab = ttk.Frame(notebook)
self.courses_tab = ttk.Frame(notebook)
self.register_tab = ttk.Frame(notebook)
self.report_tab = ttk.Frame(notebook)

notebook.add(self.students_tab, text="Students")
notebook.add(self.courses_tab, text="Courses")
notebook.add(self.register_tab, text="Register / Drop")
notebook.add(self.report_tab, text="Report")

self._build_students_tab()
self._build_courses_tab()
self._build_register_tab()
self._build_report_tab()
self._refresh_id_dropdowns()

self._refresh_students_table()
self._refresh_courses_table()

# ---- helpers -----
def _error(self, err):
    messagebox.showerror("Error", str(err))

def _success(self, msg):
    messagebox.showinfo("Success", msg)

```

```

def _clear_entries(self, *entries):
    for e in entries:
        e.delete(0, tk.END)

# ---- Students tab -----
def _build_students_tab(self):
    frame = self.students_tab
    form = ttk.LabelFrame(frame, text="Add Student")
    form.pack(fill="x", padx=10, pady=10)

    ttk.Label(form, text="Student ID").grid(row=0, column=0, padx=5, pady=5, sticky="e")
    self.sid_entry = ttk.Entry(form)
    self.sid_entry.grid(row=0, column=1, padx=5, pady=5)

    ttk.Label(form, text="Name").grid(row=0, column=2, padx=5, pady=5, sticky="e")
    self.sname_entry = ttk.Entry(form)
    self.sname_entry.grid(row=0, column=3, padx=5, pady=5)

    ttk.Label(form, text="Email").grid(row=0, column=4, padx=5, pady=5, sticky="e")
    self.semail_entry = ttk.Entry(form)
    self.semail_entry.grid(row=0, column=5, padx=5, pady=5)

    ttk.Button(form, text="Add Student", command=self._add_student).grid(
        row=0, column=6, padx=10, pady=5
    )

    columns = ("id", "name", "email", "credits")
    self.students_table = ttk.Treeview(frame, columns=columns, show="headings", height=15)
    headings = {"id": "Student ID", "name": "Name", "email": "Email", "credits": "Credits"}
    for col in columns:
        self.students_table.heading(col, text=headings[col])
        self.students_table.column(col, width=150, anchor="center")
    self.students_table.pack(fill="both", expand=True, padx=10, pady=(0, 10))

def _add_student(self):
    try:
        student = self.system.add_student(
            self.sid_entry.get(), self.sname_entry.get(), self.semail_entry.get()
        )
        self.system.save(self.DATA_FILE)
        self._success(f"Student added: {student.student_id} - {student.name}")
        self._clear_entries(self.sid_entry, self.sname_entry, self.semail_entry)
        self._refresh_students_table()
        self._refresh_id_dropdowns()
    except RegistrationError as e:
        self._error(e)

```

```

def _refresh_students_table(self):
    self.students_table.delete(*self.students_table.get_children())
    for s in self.system.students:
        self.students_table.insert(
            "", "end", values=(s.student_id, s.name, s.email, f"{s.total_credits}/{Student.MAX_CREDITS}")
        )

# ---- Courses tab -----
def _build_courses_tab(self):
    frame = self.courses_tab
    form = ttk.LabelFrame(frame, text="Add Course")
    form.pack(fill="x", padx=10, pady=10)

    ttk.Label(form, text="Code").grid(row=0, column=0, padx=5, pady=5, sticky="e")
    self.cid_entry = ttk.Entry(form)
    self.cid_entry.grid(row=0, column=1, padx=5, pady=5)

    ttk.Label(form, text="Title").grid(row=0, column=2, padx=5, pady=5, sticky="e")
    self.ctitle_entry = ttk.Entry(form)
    self.ctitle_entry.grid(row=0, column=3, padx=5, pady=5)

    ttk.Label(form, text="Credits").grid(row=0, column=4, padx=5, pady=5, sticky="e")
    self.ccredits_entry = ttk.Entry(form, width=6)
    self.ccredits_entry.grid(row=0, column=5, padx=5, pady=5)

    ttk.Label(form, text="Capacity").grid(row=0, column=6, padx=5, pady=5, sticky="e")
    self.ccapacity_entry = ttk.Entry(form, width=6)
    self.ccapacity_entry.grid(row=0, column=7, padx=5, pady=5)

    ttk.Button(form, text="Add Course", command=self._add_course).grid(
        row=0, column=8, padx=10, pady=5
    )

    columns = ("code", "title", "credits", "seats")
    self.courses_table = ttk.Treeview(frame, columns=columns, show="headings", height=15)
    headings = {"code": "Code", "title": "Title", "credits": "Credits", "seats": "Seats Available"}
    for col in columns:
        self.courses_table.heading(col, text=headings[col])
        self.courses_table.column(col, width=150, anchor="center")
    self.courses_table.pack(fill="both", expand=True, padx=10, pady=(0, 10))

def _add_course(self):
    try:
        credits_text = self.ccredits_entry.get().strip()
        capacity_text = self.ccapacity_entry.get().strip()
        if not credits_text.isdigit() or not capacity_text.isdigit():
            raise RegistrationError("Credits and capacity must be whole numbers.")
        course = self.system.add_course(

```

```

        self.cid_entry.get(), self.ctitle_entry.get(), int(credits_text), int(capacity_text)
    )
    self.system.save(self.DATA_FILE)
    self._success(f"Course added: {course.code} - {course.title}")
    self._clear_entries(self.cid_entry, self.ctitle_entry, self.ccredits_entry, self.ccapacity_entry)
    self._refresh_courses_table()
    self._refresh_id_dropdowns()
except RegistrationError as e:
    self._error(e)

def _refresh_courses_table(self):
    self.courses_table.delete(*self.courses_table.get_children())
    for c in self.system.courses:
        self.courses_table.insert(
            "", "end", values=(c.code, c.title, c.credits, f"{c.seats_available}/{c.capacity}")
        )

# ---- Register / Drop tab -----
def _build_register_tab(self):
    frame = self.register_tab
    form = ttk.LabelFrame(frame, text="Register or Drop a Course")
    form.pack(fill="x", padx=10, pady=10)

    ttk.Label(form, text="Student").grid(row=0, column=0, padx=5, pady=8, sticky="e")
    self.reg_student_cb = ttk.Combobox(form, state="readonly", width=30)
    self.reg_student_cb.grid(row=0, column=1, padx=5, pady=8)

    ttk.Label(form, text="Course").grid(row=0, column=2, padx=5, pady=8, sticky="e")
    self.reg_course_cb = ttk.Combobox(form, state="readonly", width=35)
    self.reg_course_cb.grid(row=0, column=3, padx=5, pady=8)

    ttk.Button(form, text="Register", command=self._do_register).grid(row=0, column=4, padx=5)
    ttk.Button(form, text="Drop", command=self._drop).grid(row=0, column=5, padx=5)

    self._refresh_id_dropdowns()

    info = ttk.LabelFrame(frame, text="Selected Student's Courses")
    info.pack(fill="both", expand=True, padx=10, pady=(0, 10))
    columns = ("code", "title", "credits")
    self.student_courses_table = ttk.Treeview(info, columns=columns, show="headings", height=12)
    for col, label in zip(columns, ("Code", "Title", "Credits")):
        self.student_courses_table.heading(col, text=label)
        self.student_courses_table.column(col, width=150, anchor="center")
    self.student_courses_table.pack(fill="both", expand=True, padx=5, pady=5)

    self.reg_student_cb.bind("<<ComboboxSelected>>", lambda e: self._refresh_selected_student_courses())

def _refresh_id_dropdowns(self):

```

```

student_values = [f'{s.student_id} - {s.name}' for s in self.system.students]
course_values = [f'{c.code} - {c.title}' for c in self.system.courses]
self.reg_student_cb["values"] = student_values
self.reg_course_cb["values"] = course_values
# The Report tab's dropdown doesn't exist yet the first time this runs
# (it's built after the Register tab), so only refresh it once it's there.
if hasattr(self, "report_student_cb"):
    self.report_student_cb["values"] = student_values

def _selected_id(self, combobox):
    value = combobox.get()
    if not value:
        raise RegistrationError("Please select a value first.")
    return value.split(" - ")[0]

def _do_register(self):
    try:
        sid = self._selected_id(self.reg_student_cb)
        code = self._selected_id(self.reg_course_cb)
        student, course = self.system.register_course(sid, code)
        self.system.save(self.DATA_FILE)
        self._success(
            f'{student.name} registered for {course.code}. '
            f'Total credits: {student.total_credits}/{Student.MAX_CREDITS}'
        )
        self._refresh_students_table()
        self._refresh_courses_table()
        self._refresh_selected_student_courses()
    except RegistrationError as e:
        self._error(e)

def _drop(self):
    try:
        sid = self._selected_id(self.reg_student_cb)
        code = self._selected_id(self.reg_course_cb)
        student, course = self.system.drop_course(sid, code)
        self.system.save(self.DATA_FILE)
        self._success(
            f'{student.name} dropped {course.code}. '
            f'Total credits: {student.total_credits}/{Student.MAX_CREDITS}'
        )
        self._refresh_students_table()
        self._refresh_courses_table()
        self._refresh_selected_student_courses()
    except RegistrationError as e:
        self._error(e)

def _refresh_selected_student_courses(self):
    self.student_courses_table.delete(*self.student_courses_table.get_children())

```

```

value = self.reg_student_cb.get()
if not value:
    return
sid = value.split(" - ")[0]
try:
    student = self.system._get_student(sid)
except RegistrationError:
    return
for c in student.courses:
    self.student_courses_table.insert("", "end", values=(c.code, c.title, c.credits))

# ---- Report tab -----
def _build_report_tab(self):
    frame = self.report_tab
    form = ttk.LabelFrame(frame, text="Generate Registration Report")
    form.pack(fill="x", padx=10, pady=10)

    ttk.Label(form, text="Student").grid(row=0, column=0, padx=5, pady=8, sticky="e")
    self.report_student_cb = ttk.Combobox(form, state="readonly", width=30)
    self.report_student_cb.grid(row=0, column=1, padx=5, pady=8)

    ttk.Button(form, text="Generate Report", command=self._generate_report).grid(
        row=0, column=2, padx=10
    )

    self.report_text = tk.Text(frame, height=20, font=("Courier New", 10))
    self.report_text.pack(fill="both", expand=True, padx=10, pady=(0, 10))
    self.report_text.configure(state="disabled")

def _generate_report(self):
    try:
        value = self.report_student_cb.get()
        if not value:
            raise RegistrationError("Please select a student first.")
        sid = value.split(" - ")[0]
        student = self.system._get_student(sid)

        lines = []
        lines.append("=" * 60)
        lines.append("REGISTRATION REPORT")
        lines.append("=" * 60)
        lines.append(f"Student ID : {student.student_id}")
        lines.append(f"Name      : {student.name}")
        lines.append(f>Email     : {student.email}")
        lines.append("-" * 60)
        lines.append(f"{'Code':<10} {'Course Title':<32} {'Credits':>8}")
        lines.append("-" * 60)
        if student.courses:

```

```

        for c in student.courses:
            lines.append(f" {c.code:<10} {c.title:<32} {c.credits:>8}")
        else:
            lines.append("No courses registered.")
    lines.append("-" * 60)
    lines.append(f"{'Total Credits':<42} {student.total_credits:>8}")
    lines.append(f"{'Credit Limit':<42} {Student.MAX_CREDITS:>8}")
    lines.append(f"{'Remaining Credits':<42} {Student.MAX_CREDITS - student.total_credits:>8}")
    lines.append("=" * 60)

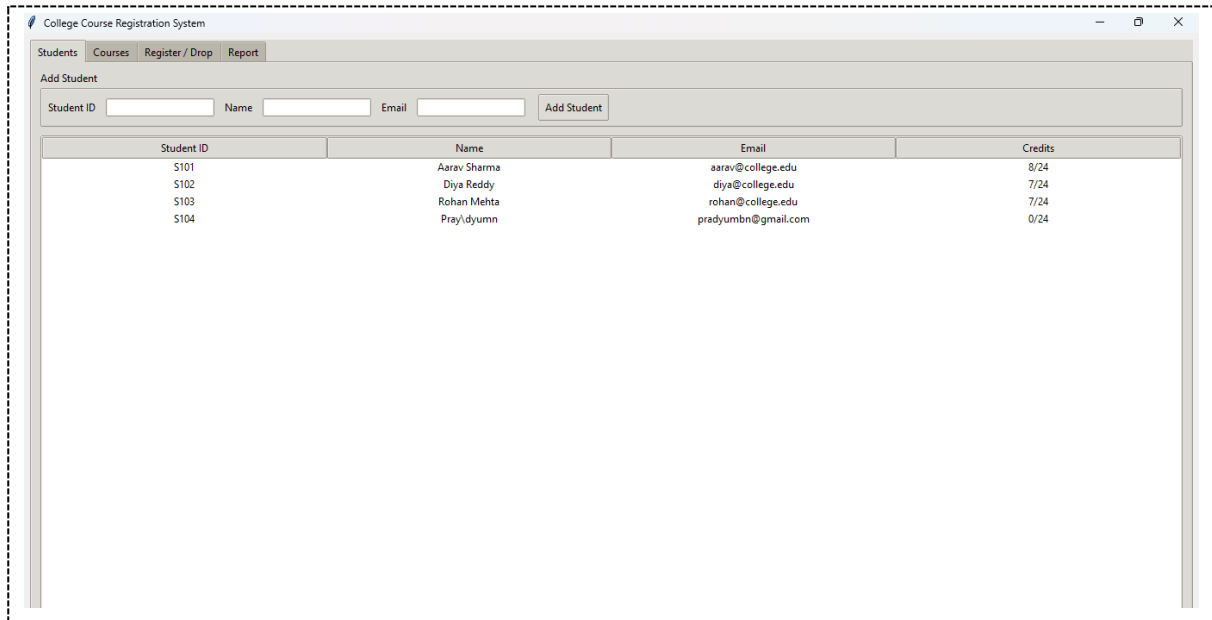
    self.report_text.configure(state="normal")
    self.report_text.delete("1.0", tk.END)
    self.report_text.insert(tk.END, "\n".join(lines))
    self.report_text.configure(state="disabled")
except RegistrationError as e:
    self._error(e)

if __name__ == "__main__":
    app = RegistrationApp()
    app.mainloop()

```

5. Results and Discussion

The application was run with the sample data and the operations of each tab were tested. Figures 3 to 7 show the output of the main screens. The behaviour of the registration rules was also checked with a series of test cases, whose results are listed in Table 4.



The screenshot displays the 'College Course Registration System' window. The 'Students' tab is selected, showing an 'Add Student' form at the top with fields for Student ID, Name, and Email, and an 'Add Student' button. Below the form is a table listing four sample students with their IDs, names, emails, and credits.

Student ID	Name	Email	Credits
S101	Aarav Sharma	aarav@college.edu	8/24
S102	Diya Reddy	diya@college.edu	7/24
S103	Rohan Mehta	rohan@college.edu	7/24
S104	Pray/dyumn	pradyumn@gmail.com	0/24

Figure 3: Output (Students tab with the sample students and their credits)

College Course Registration System				
Students Courses Register / Drop Report				
Add Course				
Code	Title	Credits	Capacity	Add Course
Code	Title	Credits	Seats Available	
CS101	Introduction to Programming	4	1/3	
CS201	Data Structures	4	1/2	
CS301	Database Systems	3	30/30	
MA101	Calculus I	4	40/40	
PH101	Physics I	4	35/35	
EN101	Technical Communication	2	50/50	
CS401	Advanced Algorithms	6	25/25	
CS402	Machine Learning	6	25/25	
CS50	Hardware C	4	20/20	
AI101	Artificial Intelligence	4	20/20	
ECE101	Digital Electronics	3	23/23	
CS501	Computer Organisation	4	20/20	
CS901	Operating Systems	3	15/15	
CH101	Engineering Chemistry	3	19/20	
CS801	Computer Networks	3	20/20	
CS601	Big Data Analytics	4	20/20	
CS600	Python Programming	1	20/20	
CS602	Software Engineering	3	25/25	
CS603	Database Security	3	18/18	
CS604	Cloud Computing	4	24/24	
CS605	Data Mining	3	20/20	
EC401	VLSI Design	4	23/24	
EC402	Embedded Systems	4	26/26	
EC403	Wireless Communication	3	20/20	
EC404	Digital Signal Processing	4	22/22	
EE301	Control Systems	4	22/22	
EE302	Renewable Energy	3	17/18	
ME301	Manufacturing Processes	3	20/20	
ME302	Machine Design	4	22/22	
CE301	Geotechnical Engineering	4	20/20	
CE302	Hydraulics and Hydrology	4	20/20	

Figure 4: Output (Courses tab with seats available)

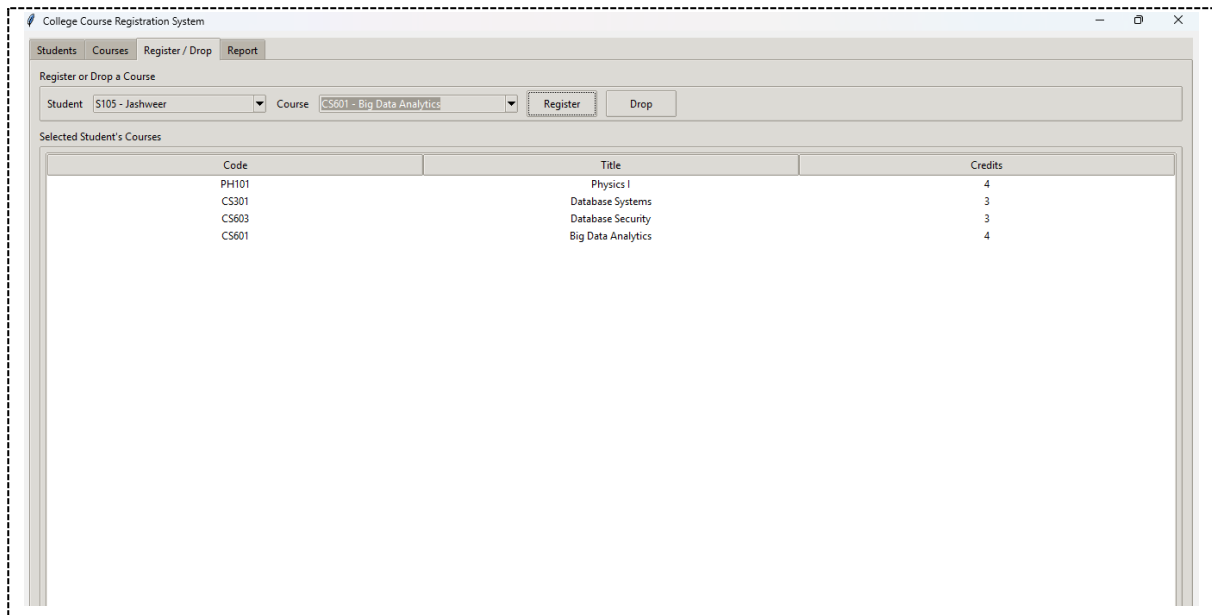


Figure 5: Output (Register / Drop tab - successful registration)

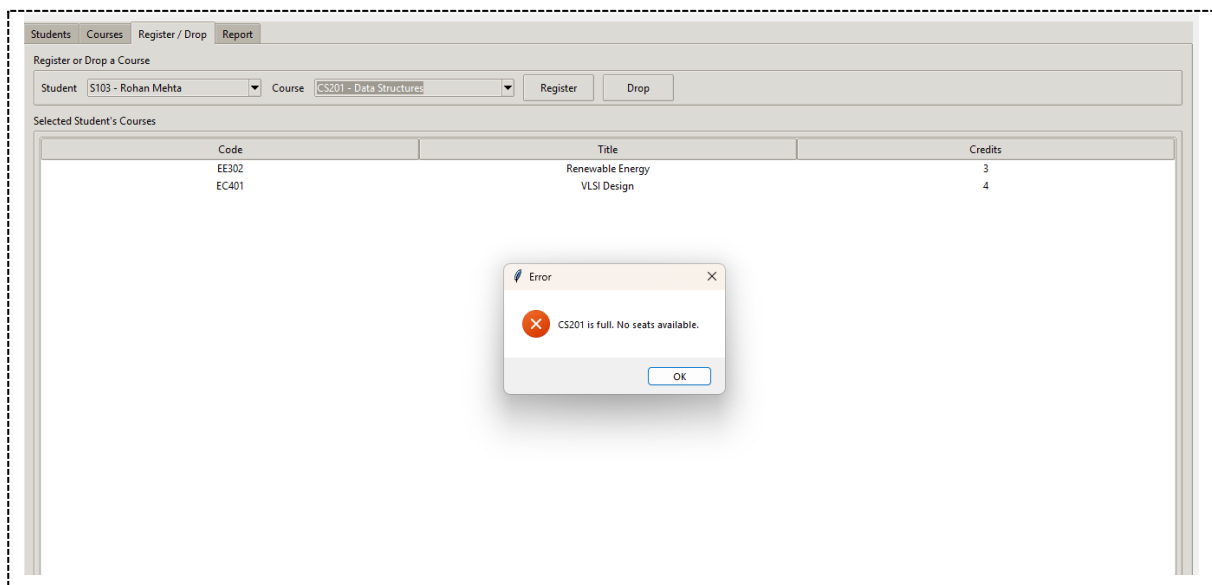


Figure 6: Output (Error messages - course full / credit limit exceeded)

College Course Registration System

Students Courses Register / Drop Report

Generate Registration Report

Student: S105 - Jashveer Generate Report

REGISTRATION REPORT

Student ID : S105
 Name : Jashveer
 Email : jashveer@gmail.com

Code	Course Title	Credits
PH101	Physics I	4
CS301	Database Systems	3
CS603	Database Security	3
CS601	Big Data Analytics	4
CS201	Data Structures	4

Total Credits 18
 Credit Limit 24
 Remaining Credits 6

Figure 7: Output (Report tab - registration report of a student)

Table 4: Test cases and system responses (starting from the sample data)

No.	Operation	Input	System response
1	Register	S103 → CS101	Accepted. S103 now has 4 credits; CS101 has no seats left (3/3).
2	Register (course full)	New student S104 → CS101	CS101 is full. No seats available.
3	Register (duplicate)	S101 → CS101	Aarav Sharma is already registered for CS101.
4	Register	S102 → CS201	Accepted. CS201 is now full (2/2); S102 has 8 credits.
5	Register (course full)	S103 → CS201	CS201 is full. No seats available.
6	Register three courses	S101 → CS401, CS402, CS505	All accepted. Credits grow to 14, 20 and 24.
7	Register (credit limit)	S101 → CS504 (3 credits)	Credit limit exceeded. Current: 24, Course: 3, Max allowed: 24.
8	Drop	S101 drops CS505	Accepted. S101 now has 20 credits.
9	Drop (not registered)	S103 drops CS201	Rohan Mehta is not registered for CS201.
10	Unknown ID	S999 → CS101	No student found with ID 'S999'.
11	Add student (bad e-mail)	S105, e-mail "bad"	Please enter a valid email address.
12	Add student (duplicate ID)	S101	Student ID 'S101' already exists.

No.	Operation	Input	System response
13	Add course (invalid credits)	CS900, credits = 0	Credits must be a positive number.
14	Save and reload	Save, then load the file	Same number of students, courses and registrations restored.

Summary of Results

The system behaved correctly in all the test cases. Valid registrations were accepted and both the student and the course were updated together; every violation of a rule was rejected with a clear message and left the data unchanged. The tables on the Students and Courses tabs refreshed automatically after each action, and the data saved in the JSON file was restored correctly on reload.

Interpretation

The results show that placing the rules inside the Student, Course and RegistrationSystem classes, instead of in the GUI code, makes the behaviour consistent: the same checks apply no matter which part of the interface triggers the action. Using one custom exception allowed the GUI to handle all errors in a single try/except block. The atomic save and the tolerant loader keep the stored data safe even if the program is closed unexpectedly or the file is edited by hand.

Achievement of Objectives

The project met its objectives: an object-oriented registration system was built with encapsulated classes and a custom exception, a four-tab Tkinter interface was provided for all operations, and the credit, capacity and duplicate rules were enforced with data stored persistently in a JSON file.

Implications

The application can be used in academic settings to teach object-oriented design and GUI programming, and as a small registration tool for a department, club or training programme. Because the logic is independent of the interface, the same classes could be reused behind a web interface or connected to a database in the future.

6. Mapping POs, PSOs and SDGs

6.1 Program Outcomes (POs) Mapping

Table 5: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied object-oriented programming, data structures (lists and dictionaries), exception handling and file handling in Python to build a working registration system.
PO2	Problem Analysis	Analysed the problems of manual course registration (over-booking, duplicate entries, excess credits) and converted them into precise rules and classes.
PO3	Design/Development of Solutions	Designed a layered solution with encapsulated classes, input validation, persistent storage and a Tkinter GUI that is separate from the logic.
PO4	Conduct Investigations of Complex Problems	Tested the system with different scenarios such as full courses, duplicate registrations, credit overflow and invalid input, and verified the results.
PO5	Engineering Tool Usage	Used Python, the Tkinter/ttk toolkit, the json and pathlib modules and a code editor to develop and run the application.
PO8	Individual and Collaborative Team work	Shared the design, coding, testing and documentation work among team members and reviewed each other's contributions.
PO9	Communication	Presented the design, results and findings through diagrams, GUI output and this report; the GUI itself communicates clearly through messages.
PO11	Life-Long Learning	Learned GUI programming, JSON handling and safe file-writing techniques beyond the classroom through self-study and practice.

6.2 Program Specific Outcomes (PSOs) Mapping

Table 6: Mapping of Program Specific Outcomes (PSOs) to Project Relevance

POS No.	Program Specific Outcome	Relevance
POS1	Software Development and Problem Solving	Applied core computer science concepts (data structures, object-oriented design, file handling and exception handling) to design and implement a complete, working software application in Python.
POS2	Application and Interface Development	Built a user-friendly desktop application with a Tkinter GUI, input validation and persistent JSON storage, following good practices such as separating logic from the interface.

POS3	Professional Practice and Continuous Learning	Used standard development practices (modular classes, testing of business rules, documentation) and explored new libraries and techniques independently to complete the project.
------	---	--

6.3 Sustainable Development Goals (SDGs) Mapping

Table 6: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 4	Quality Education	Makes administration of courses easier and more transparent, and serves as a learning example of object-oriented design and GUI programming.
SDG 8	Decent Work and Economic Growth	Builds software development and problem-solving skills that improve employability in IT and technology-driven industries.
SDG 9	Industry, Innovation and Infrastructure	Promotes computational thinking and innovation in software design, and supports the digitization of institutional processes.

7. Conclusion

This project successfully implemented a College Course Registration System in Python using an object-oriented design and a Tkinter graphical interface. The system manages students and courses, allows registration and dropping of courses, and generates a registration report, while automatically enforcing the 24-credit limit, the seat capacity of each course and the ban on duplicate registration. Custom exceptions, input validation and JSON-based persistence make the application reliable, and the separation of logic from the interface keeps the code easy to understand and extend.

For future enhancements, the JSON file can be replaced by a database such as SQLite or MySQL to support many users and larger data. Features such as login for students and administrators, course prerequisites, timetable clash detection, waiting lists, search and sorting, editing and deleting records, and export of reports to PDF or CSV would make the tool closer to a real institutional system. Automated unit tests for the logic classes and a web-based version would further improve its quality and accessibility.

Bibliography

Online Resources

- [1] Python Software Foundation – tkinter: Python interface to Tcl/Tk.
<https://docs.python.org/3/library/tkinter.html>
- [2] Python Software Foundation – tkinter.ttk: Tk themed widgets.
<https://docs.python.org/3/library/tkinter.ttk.html>
- [3] Python Software Foundation – tkinter: Python interface to Tcl/Tk.
<https://docs.python.org/3/library/tkinter.html>
- [4] Python Software Foundation – json: JSON encoder and decoder.
<https://docs.python.org/3/library/json.html>
- [5] Python Software Foundation – pathlib: Object-oriented filesystem paths.
<https://docs.python.org/3/library/pathlib.html>

Books and references

- [1] Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
- [2] Matthes, E. (2023). Python Crash Course (3rd ed.). No Starch Press.
- [3] Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.
- [4] Booch, G., Maksimchuk, R. A., Engle, M. W., Young, B. J., Conallen, J., & Houston, K. A. (2007). Object-Oriented Analysis and Design with Applications (3rd ed.). Addison-Wesley.
- [5] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education